

ABD – EXAMEN 1 (RESUMEN)

IKER FERNÁNDEZ MOLANO

ÍNDICE

1. GESTIÓN USUARIOS	3
2. GESTIÓN DE PRIVILEGIOS.....	4
3. ROLES	6
4. VISTAS	7
5. PROCEDIMIENTOS ALMACENADOS.....	8
6. LOGS.....	9
7. TRIGGERS	10
8. ÍNDICES.....	11
8.1. ÍNDICES PARA TEXTO.....	12
9. PROCESAMIENTO.....	13
9.1. ESQUEMA DE RENDIMIENTO	14
10. PARTICIONADO	14
11. FUNCIONES VENTANA.....	15
12. CONSULTAS ANIDADAS	16
13. JSON.....	17
14. GIS	18

1. GESTIÓN USUARIOS

Crear usuario:

```
CREATE USER '<nombre>@'<host> (localhost)' IDENTIFIED BY  
'contraseña';
```

Borrar usuario:

```
DROP USER '<nombre>@'<host>';
```

2. GESTIÓN DE PRIVILEGIOS

Dar privilegios a usuario:

```
GRANT <privilegio(s)> ON <db>.<tabla> TO '<nombre>'@'<host>' (WITH  
GRANT OPTION);
```

** WITH GRANT OPTION permite la propagación del permiso.*

Quitar privilegios a usuario:

```
REVOKE (GRANT OPTION FOR) <privilegio(s)> ON <db>.<tabla> FROM  
'<nombre>'@'<host>';
```

Privilegios habituales:

- CREATE
- INSERT
- SELECT
- DROP
- REFERENCES
- ALTER
- CREATE ROUTINE
- EXECUTE
- CREATE VIEW
- CREATE TRIGGER
- ALL
- RELOAD (FLUSH PRIVILEGES)
- SUPER
- UPDATE

Listado completo en: <https://dev.mysql.com/doc/refman/8.4/en/privileges-provided.html#privileges-provided-summary>

** Para roles en vez del usuario se pone el nombre del rol*

Mostrar permisos:

- Usuario actual:

```
SHOW GRANTS;
```

- Cualquier usuario:

`SHOW GRANTS FOR 'nombre'@'<host>';`

* Para roles se añade `USING <nombre-rol>`

3. ROLES

Crear rol:

```
CREATE ROLE <nombre>;
```

Borrar rol:

```
DROP <nombre>;
```

Establecer rol:

```
SET ROLE <nombre>;
```

Establecer rol por defecto:

```
SET DEFAULT ROLE <nombre> TO '<nombre>'@'<host>';
```

Mostrar rol actual:

```
SELECT current_role();
```

4. VISTAS

Crear vista:

```
CREATE VIEW <db>.<nombre-vista> AS <select>;
```

Borrar vista:

```
DROP VIEW <nombre-vista>;
```


5. PROCEDIMIENTOS ALMACENADOS

Cambiar delimitador:

```
DELIMITER //
```

Crear procedimiento:

```
CREATE PROCEDURE <nombre> ((IN/OUT) <parámetros> (TIPO))  
BEGIN  
<sentencias>  
END //
```

Dar valor a valores:

```
SELECT <columna> INTO <nombre-valor> FROM ... WHERE ...;  
SET <nombre-valor> = <valor>
```

Declarar variables:

```
DECLARE <variable> <tipo>;
```

Estructuras:

```
IF <condición> THEN  
ELSE
```

```
WHILE <condición> DO  
END WHILE
```

Mostrar procedimientos existentes:

```
SHOW PROCEDURE STATUS WHERE DB='<db>';
```

Llamar a un procedimiento:

```
CALL <nombre>(<parámetros>);
```

6. LOGS

Mostrar estado:

```
SELECT @@general_log;
```

Mostrar ubicación:

```
SELECT @@general_log_file;
```

Activar/Desactivar:

```
SET GLOBAL general_log='ON/OFF';
```

7. TRIGGERS

Crear trigger:

```
CREATE TRIGGER <nombre> <cuándo> <acción> ON <tabla> FOR EACH ROW  
<acciones-trigger>
```

- <cuándo>: BEFORE – AFTER
- <acción>: INSERT – DELETE – UPDATE

* NEW hace referencia a la nueva fila en INSERT

* Se pueden realizar varias acciones como en los procedimientos con BEGIN ... END

Borrar trigger:

```
DROP TRIGGER <nombre>;
```

Mostrar triggers:

```
SHOW TRIGGERS;
```

8. ÍNDICES

Crear índice:

```
ALTER TABLE <tabla> ADD INDEX <nombre>(<columna(s)>);
```

Borrar índice:

```
ALTER TABLE <tabla> DROP INDEX <nombre>;
```

Mostrar índices:

- Tabla concreta:

```
SHOW INDEX FROM <tabla>;
```
- Todas las tablas de la DB:

```
SELECT <tabla>, <índice>, <tipo-índice> FROM  
INFORMATION_SCHEMA.STATISTICS WHERE TABLE_SCHEMA = "<db>";
```

Mostrar espacio que ocupan los índices:

```
SHOW TABLE STATUS FROM <db>;
```

Medir tiempo de consultas:

```
SET profiling=1;  
SHOW profiles;
```

Activar/Desactivar índice:

```
ALTER TABLE <tabla> ALTER INDEX <nombre-índice> VISIBLE/INVISIBLE;
```

8.1. ÍNDICES PARA TEXTO

Crear índice:

```
ALTER TABLE <tabla> ADD FULLTEXT INDEX <nombre>(<columna(s)>);
```

Borrar índice:

```
ALTER TABLE <tabla> DROP INDEX <nombre>;
```

Consulta con índice:

```
SELECT <columna(s)> FROM <tabla> WHERE MATCH(<col1>,...) AGAINST  
(<expresión>)
```

In boolean mode:

* Añadir - delante de una palabra es como un NOT.

* Añadir * tras una palabra incluye sus derivadas.

9. PROCESAMIENTO

Estadísticas sobre los índices:

```
SELECT * FROM information_schema.statistics WHERE  
table_schema="<db>";
```

Mostrar plan de ejecución NO ejecuta consulta:

```
EXPLAIN (<formato>) <consulta>;
```

Mostrar plan de ejecución SÍ ejecuta consulta:

```
EXPLAIN ANALYZE <consulta>;
```

9.1. ESQUEMA DE RENDIMIENTO

Mostrar estado:

```
SHOW VARIABLES LIKE 'performance_schema';
```

Activar la recopilación de información de rendimiento:

```
UPDATE performance_schema.setup_instruments SET ENABLED = 'YES',  
TIMED = 'YES';  
UPDATE performance_schema.setup_consumers SET ENABLED = 'YES';
```

Obtener ID de la consulta:

```
SELECT EVENT_ID, TRUNCATE(TIMER_WAIT/1000000000000,6) as Duration,  
SQL_TEXT FROM performance_schema.events_statements_history_long  
WHERE SQL_TEXT like '%<parte-de-la-consulta>%';
```

Mostrar información de rendimiento:

```
SELECT event_name AS Stage, TRUNCATE(TIMER_WAIT/1000000000000,6) AS  
Duration FROM performance_schema.events_stages_history_long WHERE  
NESTING_EVENT_ID=<id-consulta>;
```

10. PARTICIONADO

Crear tabla con particionado, ejemplo:

```
CREATE TABLE people (  
    id INT NOT NULL,  
    age INT NOT NULL,  
    PRIMARY KEY (id, age)  
)  
PARTITION BY RANGE COLUMNS (age) (  
    PARTITION p1 VALUES LESS THAN (10),  
    PARTITION p2 VALUES LESS THAN (18),  
    PARTITION p3 VALUES LESS THAN (MAXVALUE)  
);
```

Añadir una nueva partición:

```
ALTER TABLE ... ADD PARTITION(PARTITION ... VALUES LESS THAN(...));
```

Borrar una partición:

```
ALTER TABLE ... DROP PARTITION ...;
```

Reorganizar una partición:

```
ALTER TABLE ... REORGANIZE PARTITION ... INTO(  
    PARTITION ... VALUES LESS THAN(...),  
    ...  
);
```

11. FUNCIONES VENTANA

Mostrar cualquier valor de una columna:

```
ANY_VALUE(<columna>);
```

Mostrar página:

```
SELECT <columna(s)>, ROW_NUMBER() OVER((PARTITION BY <columna>))  
FROM <tabla>;
```

Clasificar:

```
SELECT <columna(s)>, RANK() OVER((PARTITION BY <columna> ORDER BY  
<columna> (DESC))) FROM <tabla>;
```


12. CONSULTAS ANIDADAS

IN:

```
SELECT <columna(s)> FROM <tabla> WHERE <columna> (NOT)  
IN(<consulta>);
```

Que cumpla que para todos ALL algo:

```
SELECT <columna(s)> FROM <tabla> WHERE <columna> =><  
ALL(<consulta>);
```

Existencia:

```
SELECT <columna(s)> FROM <tabla> WHERE (NOT) EXISTS(<consulta>);
```

En la consulta 2 se puede acceder referencia a la consulta 1:

```
SELECT <columna(s)> FROM (<consulta>) AS <nombre1>,  
LATERAL(<consulta>) AS <nombre2>;
```

Número outputs:

```
LIMIT <número>;
```

A partir de que fila sacar:

```
OFFSET <número>;
```

Tabla temporal:

```
CREATE TEMPORARY TABLE <nombre>(<consulta>);
```

Expresiones comunes de tabla, CTE:

```
WITH <nombre>(<nombre-columna(s)>) AS(<consulta>) SELECT <nombre-  
columna(s)> FROM <nombre>;
```

13. JSON

Insertar:

```
INSERT INTO ... VALUES('{ "key": "value", ...}');  
INSERT INTO ... VALUES(JSON_OBJECT('key', 'value', 'key', 'value'));  
JSON_OBJECT('key', JSON_ARRAY('value1', 'value2', 'value3'))
```

Mostrar:

```
SELECT <columna>->'<valor>' FROM ... WHERE <columna>-  
><clave>='<valor>';
```

Mostrar formateados:

```
SELECT JSON_PRETTY(<columna>) FROM ...
```

Buscar:

```
SELECT <columna(s)> FROM ... WHERE JSON_SEARCH(<columna>,  
'all', "<valor>");
```

Actualizar datos:

```
UPDATE ... SET <columna>= JSON_SET(<columna>,'<clave>','<valor>')  
WHERE ...;
```

Añadir índice a un JSON:

```
ALTER TABLE ... ADD valkey <tipo> AS (doc->>'<clave>') VIRTUAL,  
ADD INDEX idx_key1(valkey);
```

14. GIS

Crear punto:

```
SET @<nombre> = ST_SRID(POINT(<latitud>, <longitud>),4326);
```

Mostrar punto:

```
SELECT ST_AsText(@<nombre>);
```

Mostrar componente:

```
ST_X/Y(@<punto>);
```

Calcular distancia:

```
ST_Distance(@<punto1>, @<punto2>);
```

Comprobar si está dentro del polígono:

```
ST_Contains(@<polígono>, @<punto>);
```